

1 第一章 谋定全局：在线系统与云计算

长期运行、通过网络为多个用户提供服务的软件系统，称为在线系统（Online System）。与本地程序不同，在线系统不会在处理完一次输入后就退出，它需要在多次请求之间维持共享状态，并协调不同用户的操作结果。

网页浏览、网上交易、在线协作和网络游戏都属于在线系统，但它们面对的首要困难并不相同，复杂度也逐步加深。浏览网页时，用户最关心服务能否随时访问、内容能否及时返回，这首先是可用与性能的问题。完成网上交易时，除了及时返回，还要保证订单和支付结果准确、不重复、不丢失，这进一步提出了状态正确性的要求。多人共同编辑文档时，几个人可能同时修改同一份内容，这些同时发生的修改称为并发；系统需要协调它们并把结果同步给所有参与者，由此引出并发控制问题。在网络游戏中，服务端不仅要每个玩家的操作作出实时裁决，还要持续维护所有玩家共享的世界状态，对状态、实时性和并发规模同时提出了高要求。

尽管场景不同，这些系统面临的根本挑战是一致的。首先，系统要持续运行，不能因为一次任务结束就退出。其次，系统要同时服务多个用户，而用户数量和请求量随时可能变化。再者，系统要在多用户之间维护共享状态，确保修改有序、结果可靠。此外，随着规模扩大，机器和网络出现局部故障会成为常态，系统需要在故障下继续提供服务。

这些挑战带来了一组相互关联的问题：请求从哪里进入、由谁处理；共享状态由谁确认、保存在哪里；资源不足时怎样扩展；组件失效后怎样恢复。把这些职责分配给不同的组件，并规定它们之间如何配合，就是在线系统的架构设计。

本章沿着资源基础、逻辑结构、评价标准、架构演进和物理落地的路线展开。首先说明云计算为什么是在线系统的资源基础，没有可按需调整的计算、存储和网络资源，架构设计就无从谈起。在此之上，我们建立在线系统的通用逻辑结构，说明一次请求通常经过哪些环节、共享状态如何维护。职责明确后，再从性能、状态正确性、可扩展性、可用性和成本五个维度建立评价标准。随后可以看到，当规模、负载和运维条件变化时，架构会沿着一条可辨识的路径逐步演进，从最小的单机闭环到单机并发，再到多机协作和平台治理。最后，我们把这些逻辑职责落回到数据中心的真实物理资源上，看看它们如何由具体的硬件和设施支撑。

学完本章，读者将获得一套可以迁移到不同在线系统的分析方法：沿一次请求的处理过程识别系统中的各项职责及其相互关系，判断一次架构调整解决了什么问题、依赖哪些条件、付出了什么代价，并用性能、状态正确性、可扩展性、可用性和成本五个维度评价设计的优劣。

1.1 云计算的重要性

在线系统一旦投入运行，就要长期面对两类不断升级的压力。

第一类压力来自请求与共享状态的交互，这是任何在线系统从开始搭建就会遇到的基本问题。大量用户同时访问时，请求不会按顺序排队到达，也不会以同样的速度完成；如果逐一来处理，快请求会被慢请求拖住，系统响应就会整体变慢。更棘手的是多个请求同时读取或修改同一份数据，比如两个用户同时下单同一件商品、两个玩家同时捡起同一个道具。若不协调它们的先后顺序，两个操作可能都读到修改前的数据而同时成功，造成超卖或道具被重复拾取。

第二类压力来自规模与运行条件的变化，它会让第一类问题进一步放大和复杂化。在线服务的用户数量和请求强度通常不是恒定的，白天高、深夜低，活动期间还可能出现突发高峰。长期按峰值准备机器会造成大量闲置，只按平均水平配置又难以承受突发流量，系统需要根据实际需求分配和回收容量，同时保留足够的余量。而当机器数量增长到一定程度，单台机器、单条网络链路甚至单个程序进程的失效都会成为常态，部署过程也可能因环境差异和操作顺序出错。系统拆到多台机器后，同一份数据还可能存在多个副本，本来就棘手的状态正确性问题，现在要跨节点解决。这时再依靠人工逐台处理故障、逐次完成发布，就会越来越难以及时响应。系统需要把交付、监测、故障实例替换和服务恢复组织成可自动重复执行的流程，用自动化替代人工判断和手动操作，才能在规模扩大后保持响应速度和操作一致性。

如果应用固定运行在少数几台机器上，部署、扩容和故障恢复又主要依赖人工，就很难同时应对这些压力。要在持续变化的条件下保持服务质量，系统需要一种新的资源组织方式。云计算的核心作用，是把计算、存储和网络资源组织成可申请、可度量、可调度和可回收的资源池。应用通过虚拟机或容器获得计算资源，通过存储服务和虚拟网络使用存储与网络资源，不必绑定某一台固定机器。平台根据需求安排资源的位置和数量，并在负载或运行状态发生变化时动态调整分配。例如，活动高峰到来时，平台可以为登录服务额外调集若干台机器承接突发流量；高峰过去后，这些机器又被回收到资源池，留给其他服务复用。

不过，资源能够由平台调度，并不表示应用能够随之迁移或扩展。如果程序、配置和状态仍然依赖某一台固定机器的本地环境，平台即使拥有空闲资源，也难以安全地迁移或扩展服务。这就要求应用本身也要**按照可复制、可调度和可恢复的方式组织**，也就是云原生的思路。服务被封装为标准化交付单元，配置和状态按照明确规则管理，平台据此持续完成部署、扩缩容、故障恢复和运行观测。简单说，在线系统描述的是需要长期提供的服务，云原生描述的是这些服务如何利用云平台进行组织和治理。

归根结底，**云计算提供的是可调整的资源 and 可自动执行的治理能力**，但它不决定系统内部的请求路径和状态维护方式。要把可调度的资源转化为可持续提供的服务，还需要明确一次请求经过哪些环节，各项职责由谁承担，以及系统怎样判断和调整运行状态。下一节据此建立在线系统的通用架构。

1.2 在线系统的通用架构

以在线协作文档为例，用户在编辑器里输入一段文字后，编辑器把这次修改连同它在文档中的位置打包成一条请求，发送给服务端；服务端校验并应用这次修改，再把更新后的内容同步给所有正在编辑的人。网页下单、游戏中发出操作等其他交互也都遵循同样的过程：客户端把用户的动作变成请求发给服务端，再等待处理结果。要解释这次交互如何完成，首先需要明确客户端与服务端分别承担什么职责。客户端负责采集用户输入并呈现结果，服务端负责校验请求、执行规则并维护共享状态。客户端可以通过缓存或预测改善交互体验，但不能绕过服务端直接决定其他参与者都要接受的结果。这种职责划分构成客户端/服务器（Client/Server，C/S）架构的基本关系。服务端内部还要继续划分，请求处理、状态维护和运行治理分别由不同部分承担。

只看服务端处理一次业务请求的过程，在线系统似乎只需要接收请求、执行计算并保存状态。要让这条处理路径持续可靠运行，系统还需要部署和调整服务实例，在故障发生时恢复服务，并持续判断各组件的运行状态。这些运行治理工作与业务处理一样，都需要计算、存储和网络资源提供支撑。要同时看清请求处理、运行治理和基础设施三部分之间的关系，用一张分层结构图会更清楚。

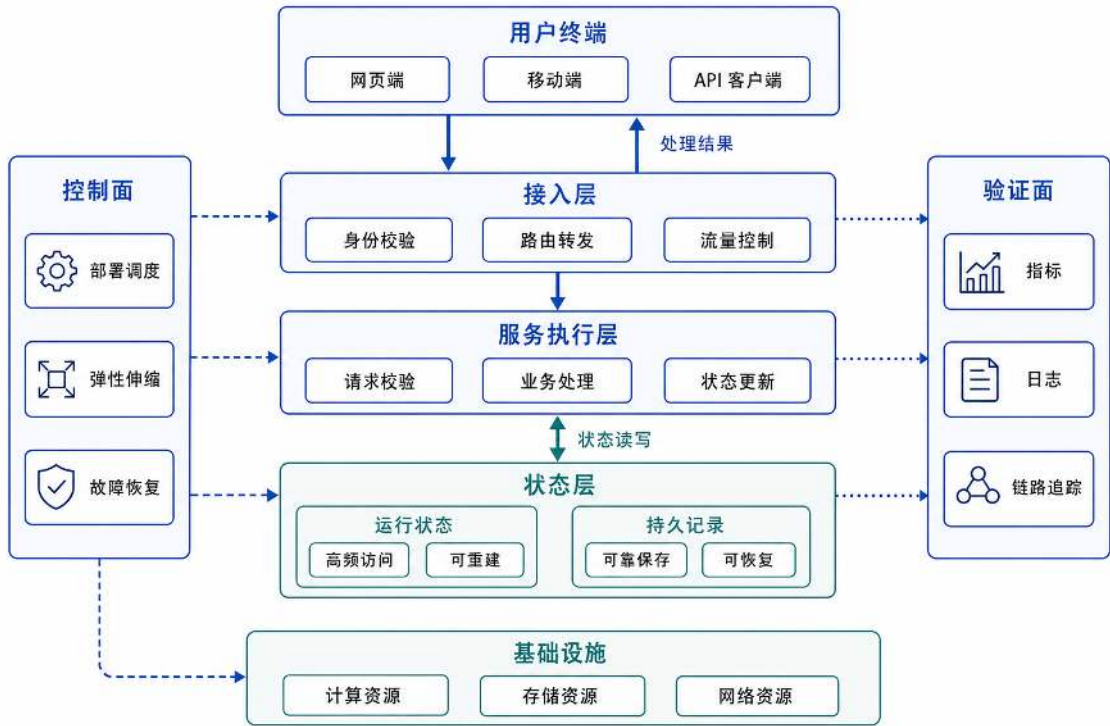


图 1: 在线系统的业务处理与运行治理：主要请求路径完成处理和状态更新，控制面与验证面维持系统运行，基础设施提供所需资源。

图 1 把系统职责组织为三个层面：沿请求路径依次排列的接入层、服务执行层和状态层，负责完成一次请求的处理和状态更新；位于两侧的控制面和验证面，分别负责调整运行方式和记录运行证据；底部的基础设施则为所有活动提供计算、存储和网络资源。下面先沿请求处理路径说明接入、执行和状态三类职责，再讨论控制面与验证面如何在负载变化和组件故障时维持这条路径。这两部分合在一起，构成在线系统通用架构的基本框架。

接入层：建立稳定且受控的系统入口 外部请求到达系统时，面对的往往是多个并行业务实例，而不是单个处理进程。以游戏为例，玩家的一次操作可能由多个战斗服务器中的某一个处理。如果客户端绕过统一入口、自己直连某个实例，这个实例就难以确认玩家身份和会话，系统也无法汇总各实例的剩余容量。更麻烦的是，突发请求可能集中涌向同一个实例，使其迅速过载。因此，系统需要在请求与业务实例之间设置一个统一入口，先判断请求是否可以进入、应该交给哪个实例、当前容量能否继续接收请求，这些职责共同构成接入层。

具体来说，接入层负责接收连接和请求，识别访问者，把请求路由到合适的服务实例，并在

系统过载或请求异常时实施限流、超时和拒绝策略。网络游戏、即时通讯属于长连接系统，客户端与服务端长时间保持连接并持续收发数据，接入层还要维护连接的生命周期；网页搜索、电商下单属于短请求系统，请求到达后很快返回，接入层则更关注请求转发、负载分配和连接复用。

接入层负责的是用户请求如何进入系统，而不是完成最终的业务处理。如果让每个业务服务各自实现身份识别、路由和限流，既重复又容易不一致；把这些职责集中到接入层统一处理，业务服务就能专注完成各自的业务逻辑。

经过接入层，外部请求被转换为身份已经识别、资源使用受到约束且去向明确的内部请求。请求能够进入系统以后，下一步才是判断它应当触发什么计算和状态变化。

服务执行层：校验请求并产生状态变化 请求进入内部后，由服务执行层完成规则校验、业务计算和结果生成。以玩家发动一次攻击为例，服务先校验请求是否合理：玩家是否拥有该技能、冷却是否结束、法力值是否足够；通过后再计算伤害和命中，并据此扣减法力值与目标生命值。如果校验和计算交给客户端自己完成，恶意玩家就能绕过限制，在法力不足时强行发动技能或篡改伤害数值。因此，只有经过服务端校验的请求才能改变共享状态，客户端提交的只是操作意图，最终结果由服务端确认。

服务执行层的设计重点是确定这些计算职责如何组合。频繁访问同一份状态的操作适合放在一起，例如伤害计算和法力扣减都要读写角色状态，合并可以避免反复跨服务取数据。资源消耗、扩展速度或故障影响差异较大的职责则可以拆开，例如战斗和聊天：前者对 CPU 要求高且需要低延迟，后者占用带宽但能容忍一定延迟，扩容节奏也不同，拆分后能各自独立扩展、故障互不牵连。不过拆分也有代价，原本一次函数调用就能完成的事，拆开就要跨网络通信并协调不同服务上的状态，因此是否拆分取决于职责差异是否大到值得付出这些成本，而不是为了形式上多分几层。

服务执行层由此把已经接入的请求转化为经过校验的处理结果和状态更新。接下来还要确定这些状态保存在哪里、保留多久，以及发生故障后如何恢复。

状态层：区分运行状态与持久记录 状态层负责保存服务执行产生的数据，但不同数据对访问速度和故障后保留的要求并不相同。会话上下文、临时计算结果和高频更新数据需要频繁读写，通常保存在内存或缓存中。账号、订单、资产和审计记录一旦丢失就难以重建，需要写入能够在进程或机器故障后保留数据的持久化存储。

如果所有更新都等待持久化存储确认，请求处理可能因写入延迟而变慢。如果所有数据都只保存在内存中，进程重启或退出后关键记录又会丢失。因此，设计状态层时需要依次回答三个问题：哪些数据丢失后可以重新计算，哪些更新必须在返回成功前持久保存，多个副本之间允许多大的状态差异。明确这些要求后，才能选择内存、缓存或数据库，并确定数据复制和故障恢复方式。经过状态层，服务执行产生的数据被分流到不同存储位置：临时状态保留在高速存储中以保证响应速度，关键记录写入持久化存储以保证故障后可恢复。

经过接入层、服务执行层和状态层，一次请求已经能够从进入系统走到结果返回和状态保存。但这只是请求路径本身通了，要让这条路径长期稳定运行，还需要另外两组能力持续维护。

控制面与验证面：让系统能够长期运行 能够完成一次请求，并不表示系统能够长期稳定运行。当实例数量、负载或运行状态发生变化时，还需要持续维护这套处理过程。这项工作由控制面和验证面共同承担：控制面回答系统应当怎样调整，验证面回答系统当前是否运行正常。

控制面负责决定服务部署到哪里、运行多少实例、哪些实例可以接收流量，以及实例异常后如何替换。规模较小时，这些决定可以由人工完成；规模扩大后，需要由控制器持续观察实际状态，并把它调整到预期状态。例如，系统声明需要运行三个服务实例，而某个实例故障后实际只剩两个，控制器就会创建替代实例，直到实际数量重新达到三个。调度、健康检查、扩缩容和生命周期管理都属于控制面的职责范围。

但是，控制器要做出这些调整，首先需要知道系统当前的运行状态，哪些实例故障了、延迟有没有升高、请求成功率是多少，这些都不能靠猜测。验证面通过指标、日志和链路记录保存运行证据，承担校验与追溯职责。系统通过这些外部信号判断内部运行状态的能力，称为可观测性 (Observability)。系统需要知道请求是否成功、延迟发生在哪一段，以及状态为何发生变化，都要从验证面获取信息。没有这些信息，故障恢复只能确认实例是否重新启动，却无法确认服务质量是否恢复。验证面既服务于排障，也为容量规划、发布检查和责任追踪提供依据。

因此，控制面和验证面是相互配合的：控制面依据验证面提供的运行证据调整实例、容量和故障恢复动作，验证面则持续记录运行状态，为调整和结果校验提供依据。两者共同依赖底层的计算、存储和网络资源，任一部分受到限制，都会影响系统的承载能力或恢复能力。

案例：一次在线会话中的职责协作 前面分别介绍了各层的职责，现在用一次完整的在线游戏会话，看各层之间如何配合。分开讨论时看到的是每层各自做什么，放在一起更关键的是层与层之间的交接：请求在什么位置从一层进入下一层、状态在谁手里被修改、运行信息又如何反馈给控制机制。

玩家进入游戏时，接入层首先建立连接、校验身份，并根据当前各实例的负载情况，把玩家会话分配到合适的服务实例。这里接入层和服务执行层之间的交接点是“已经确认身份、绑定到具体实例的会话”，后续操作请求都沿着这条已建立的通道转发。

玩家在游戏中发出移动或技能操作时，请求沿接入通道到达服务执行层。服务执行层从状态层读取当前世界状态，经过规则校验后计算新状态，再写回状态层并把权威结果返回给玩家。这一步的配合重点是，服务执行层负责规则判断，状态层负责数据存取，两者职责分离但必须协同工作——执行层不能直接绕过状态层修改数据，状态层也不会主动判断操作是否合法。

玩家退出游戏时，接入层关闭连接，服务执行层通知状态层回收会话等临时状态，同时把不能丢失的结算结果写入持久化存储。临时状态和持久记录的分流，是状态层内部两类存储之间的配合。

上述步骤构成一次在线会话的主要请求路径。在这条路径背后，控制面根据在线人数和实例状态持续调整服务容量、替换故障实例；验证面则通过指标、日志和链路记录追踪每一步的结果，为扩缩容决策和故障定位提供依据。玩家经历的会话步骤与系统内部各层的职责对应关系，如图 2 所示。

玩家感知到的接入、行动、交互和退出四个阶段，每一步背后都有接入层、执行层和状态层



图 2: 一次在线会话中的职责协作：接入层建立可信通道，服务执行层校验操作并产生结果，状态层更新共享状态并保存关键记录。

的协同：玩家看到的是操作结果，系统内部则完成了身份校验、规则裁决、状态更新和持久保存等一系列工作。在线游戏只是说明上述职责如何协作的一个具体例子。电商平台、在线协作系统和模型服务虽然处理的对象不同，同样需要通过接入层、服务执行层和状态层完成请求处理，并通过控制面与验证面维持系统运行。这组职责及其判断标准可以迁移到不同业务。

明确了在线系统需要承担的职责，接下来的问题是怎样判断一种组织方式好不好。具体架构设计需要在多种实现方式之间选择，而选择之前首先要明确评价标准。

1.3 架构设计的评价目标

能够完成一次请求，只能说明系统中的各项职责可以配合工作。在用户较少、组件正常运行时，把接入、计算和状态访问集中在一个服务中，或者分别部署为多个服务，都可能及时返回正确结果。请求增多、状态被并发修改或组件失效以后，两种组织方式的差异才会显现。评价架构不能只看正常条件下的单次请求，还要逐级加压，检查运行条件变化后系统能否保持预期行为。

第一道压力来自请求数量。当并发请求增多时，系统还能不能在可接受的时间内完成足够多的工作，这是性能要回答的问题。第二道压力来自并发修改。多个操作同时读写同一份状态时，结果是否仍然符合规则，这涉及状态正确性。第三道压力来自负载的持续变化。用户量上涨时系统能不能扩容、退潮时能不能缩容，这考验可扩展性。第四道压力来自组件故障。当机器、进程或链路突然失效时，服务还能不能继续提供，以及多久能够恢复，这取决于可用性。最后，以上所有能力的实现都需要消耗资源和运维投入，因此成本也必须纳入评估。

由此，架构设计可以从性能、状态正确性、可扩展性、可用性和成本五个维度评价。前四个维度衡量系统在不同压力下能做到什么，成本则衡量做到这些需要付出多少。下面以在线游戏在高负载、并发状态更新、容量变化和组件故障下的表现为例，说明这些维度如何约束架构选择。

1.3.1 性能：控制延迟并提高有效吞吐

性能是用户对在线系统最直接的体感：页面能否及时打开、点击能否及时响应、数据能否及时加载，都取决于它。当更多请求同时到达时，系统还能否在可接受的时间内完成足够多的工作，是性能要回答的核心问题。

例如，大型游戏活动开始后，大量玩家可能同时登录、匹配和领取奖励。系统在日常负载下响应很快，到了活动高峰，单位时间内完成的请求却不再增加，部分玩家还会长时间停留在登录或匹配页面。要解释这些现象，不能只用“快”或“慢”描述系统。

因此，性能通常从延迟和吞吐两个方面衡量。延迟表示一次请求从进入系统到得到结果所需的时间，吞吐表示单位时间内能够完成多少请求。平均延迟反映整体情况，但可能掩盖少量等待时间很长的请求。比如 100 个请求里有 95 个都在 100 毫秒内返回，但有 5 个等了 1 秒以上；平均延迟看起来还可以，但已有 5% 的请求明显偏慢。因此还需要观察 P95、P99 等尾延迟，也就是把所有请求按耗时从短到长排序后，排在第 95%、第 99% 位置的请求需要等多久。

请求较少时，CPU、网络或存储可能处于等待状态，适当增加并发可以让更多请求同时推进。随着请求到达速度接近系统的处理能力，CPU、内存、连接池或下游存储会逐渐成为瓶颈。来不及完成的请求进入队列，吞吐增长开始放缓，尾延迟和超时持续增加。

系统内的请求数量、到达速度和平均停留时间之间的关系，可以用排队论中的 **Little 定律** 更准确地描述： $L = \lambda W$ ，其中 L 是系统内平均请求数， λ 是请求到达率， W 是请求的平均停留时间。回到游戏活动的场景来看，如果每秒进来 500 个登录请求，每个请求平均停留 0.5 秒，那么系统内平均有 $500 \times 0.5 = 250$ 个请求正在处理或排队。到达速度加快或处理变慢时，平均停留时间增加，系统内的请求数也会随之增加；若到达速度持续超过处理能力，系统便不再处于稳定状态，队列会不断增长。

读者可能会以为，并发数越高系统整体越快。但当系统接近处理能力上限时，继续增加并发并不能让整体更快，反而会因为队列变长而拖慢所有请求。性能优化的目标不是尽可能提高并发数，而是在延迟和吞吐都可接受的前提下，找到合适的并发水平。

要比较不同架构的性能优劣，需要有明确的测试方法。性能测试需要明确请求到达速度、并发连接数、请求数据量、持续时间和资源配置。测试还要同时记录吞吐、尾延迟、队列长度和资源利用率，不能只测量单次请求耗时。只有测试条件一致，不同架构的性能结果才具有可比性。

性能评价不仅要看目标负载内的表现，还要看超过目标负载后的行为。当负载超过系统的处理能力时，继续接收所有请求会使队列不断扩大，并拖慢原本可以完成的请求。系统需要限制队列长度，并通过限流、超时或拒绝部分请求控制进入系统的工作量，避免局部过载演变为持续拥塞。

1.3.2 状态正确性：明确状态权威与恢复条件

性能衡量请求能否及时完成，以及单位时间内能完成多少。但响应及时并不表示状态变化一定符合业务规则。一旦请求可能并发、重试或中途失败，状态正确性就需要单独检验。客户端在一局对战结束后请求领取奖励。服务器已经发放奖励并返回成功，但响应消息在网络中丢失，客

户端随后再次发送同一请求。如果服务器把它当作新的请求处理，同一份奖励就会发放两次。如果服务器在奖励记录可靠保存之前就返回成功，进程或机器随后发生故障，玩家可能已经看到奖励，对应记录却没有保存。

除了重复请求和响应丢失，共享状态还可能被多个请求同时修改。例如，两个购买请求几乎同时读取同一份余额，都判断余额充足并完成扣款，最终结果就可能违反账户不能透支的规则。这类问题在正常的单请求测试中不会出现，却会在并发执行、网络重试和服务故障时暴露。

这些现象共同指向状态正确性问题。系统需要明确谁能够提交最终状态，以及一次更新在什么时刻算作成功。对于重复请求，还要使同一操作即使执行多次也只产生一次有效结果，这种性质称为幂等性 (Idempotency)；对于故障恢复，则要确定系统能够恢复到哪个位置。单机系统主要处理并发操作对共享数据的竞争；系统扩展到多台机器后，还要处理副本同步、跨节点更新和部分节点失效。

不同状态不必采用相同的确认方式。角色位置、在线状态等临时信息可以允许短暂滞后，资产、订单和结算记录则需要更严格的确认过程。确认过程越严格，通常意味着需要的通信和等待越多，因此状态正确性还要与延迟和可用性共同考虑。

因此，评价状态正确性不能只验证正常流程，还要主动制造并发写入、重复请求、响应丢失和进程重启等情况。验证时需要检查三类关键规则：同一结果不能重复提交，账户数据不能无故增减，已经确认的记录不能在故障后消失。

1.3.3 可扩展性：让容量随压力变化

当负载超过现有处理能力时，最直接的办法是增加机器或服务实例。但新增资源能否真正发挥作用，取决于原来的工作能否交给它处理。新赛季开放后，大量登录请求可能在短时间内集中到达，一台登录服务很快达到处理上限。此时启动第二个登录服务，并把后续请求分给它，两个实例就可以同时处理不同玩家的登录请求。

一场已经开始的对局则不同。玩家的位置、生命值和操作结果仍保存在原来的战斗服务中，新实例并不知道对局进行到了哪一步，因而不能直接接手。如果状态尚不能恢复或迁移，新实例只能承接新的对局。要让它分担已有对局，还需要提供状态恢复、会话接管和安全迁移机制。

两种场景的差异说明，新增资源能否有效分担压力，取决于工作和状态能否被新实例承接。可扩展性包含两个相互关联的方面。系统通过增加资源承载更多工作，这种能力称为扩展性。系统在负载升高时及时增加资源、在负载下降时及时回收资源，这种能力称为弹性。两者能否发挥作用，都取决于工作和状态能否被合理分配。设计时首先要明确哪部分工作需要更多资源，以及这部分工作能否交给新的实例。

给现有实例增加 CPU、内存或网络能力称为纵向扩展。这种方式实施较简单，但最终受单台机器规格限制。增加实例或节点数量称为横向扩展，可以继续扩大总资源，却要求请求和状态能够被分配到不同实例。对于维护房间、会话等状态的服务，横向扩展还要解决新请求怎样分流、已有状态是否迁移以及旧实例何时退出等问题。

增加资源也不一定带来相同比例的有效容量。如果所有实例最终都要访问同一个数据库热点、全局锁或中心节点，共享部分仍会限制整体吞吐，新增实例还可能增加连接和竞争。因此，

扩展设计需要验证原有瓶颈是否真正得到分摊，而不能只统计实例数量。

弹性还受到时间影响。扩容需要准备资源、拉取包含应用程序及其运行环境的镜像、启动服务并预热状态；缩容需要停止接收新流量、处理存量请求并保存必要状态。新增资源如果在流量高峰结束后才能投入使用，或者缩容时直接终止仍在处理请求的实例，都不能达到预期效果。

因此，评价可扩展性不能只检查实例数量是否发生变化。还要验证新增资源是否分担了实际瓶颈、是否及时投入工作，以及缩容是否在不丢失请求和状态的情况下完成。只有这些条件同时成立，容量调整才能真正改善系统在负载变化下的运行能力。

1.3.4 可用性：限定故障影响并恢复服务

可扩展性关注的是负载变化时怎样调整容量；当已有组件突然失效时，系统能否继续服务，则取决于故障隔离与恢复能力。承载对局的服务实例突然失效时，系统需要停止把新玩家送往该实例，并判断已有对局能否由其他实例接管。如果房间状态只保存在故障进程的内存中，即使立即启动新实例，也无法恢复正在进行的对局。如果系统持续保存可恢复状态并准备备用容量，恢复会更快，但数据复制和空闲资源也会增加成本。

可用性关注系统在组件失效、网络波动或版本更新期间能否继续提供服务，以及中断后需要多长时间恢复。它通常用正常服务时间占总运行时间的比例来衡量，例如 99.9% 的可用性意味着每年累计停机时间不超过约 8.76 小时，99.99% 则将年停机时间压缩到约 52 分钟。完整的恢复过程包括发现故障、停止转发新请求、切换到可用实例并恢复必要状态，缺少任何一步都可能延长服务中断时间。

副本数量不能单独证明系统可用。多个副本如果运行在同一台机器、依赖同一个网络入口或共享同一份易失状态，仍可能被一次故障同时影响。备份如果从未执行恢复验证，也不能证明关键数据能够取回。评价可用性时，需要明确准备应对哪类故障、故障会影响多少请求、多久能够发现并恢复，以及恢复后可能丢失多少状态。

1.3.5 成本：约束目标的实现方式

如果说前四个维度衡量的是系统能做到什么，成本衡量的则是做到这些需要付出多少。提高性能、状态正确性、弹性和可用性都需要付出资源与工程代价。性能优化可能增加计算和网络资源，严格确认会增加存储与通信开销，弹性需要保留可调度容量，可用性则需要副本、备份和故障演练。成本不仅包括服务器和存储费用，还包括跨节点通信、监控告警以及维护复杂系统所需的人力。

评价成本时，需要在相同服务目标下比较方案。减少冗余可以降低日常开销，却会压缩应对突发流量和故障的余量；增加冗余能够缩短恢复时间，也会提高长期成本。架构选择需要先明确哪些业务后果不能接受，再为相应目标配置能够承担的资源与运维投入。

从目标到可检验指标 以上五个维度给出了评价架构的方向，但“稳定”“快速”等描述仍然过于笼统，无法直接检验。要让评价真正可操作，需要把目标转化为可测量的指标。请求成功率、P99 延迟和恢复时间等可观测量称为服务等级指标（Service Level Indicator, SLI）。这些指标希

望长期达到的水平称为服务等级目标 (Service Level Objective, SLO)。当目标进一步形成面向用户的服务承诺时,则构成服务等级协议 (Service Level Agreement, SLA)。例如,对外提供的企业 API 可以把实际测得的请求成功率和 P99 延迟作为 SLI,把“每月请求成功率不低于 99.9%”和“P99 延迟不超过 200 毫秒”规定为 SLO;当这些目标及未达成时的补偿责任写入客户合同时,才形成 SLA。从 SLI 到 SLO 再到 SLA,架构目标被逐步转化为可以持续检查的条件。

同时必须看到,这些目标及其成本约束彼此牵制。更严格的状态确认可能增加延迟,更多副本会提高可用性但增加成本,更高的资源利用率会降低容量余量。架构设计的任务不是把某一单项指标推到最高,而是在明确业务后果后选择可以验证的平衡点。平衡点也不是一成不变的:当负载规模、状态复杂度或运维条件发生变化时,原来可行的方案可能不再满足目标,架构就需要随之调整。下一节就沿着这条思路,讨论运行条件变化如何驱动架构逐步演进。

1.4 从双人对战到云平台：架构演进的四个阶段

前两节分别说明了在线系统需要承担哪些职责,以及应当用哪些目标评价架构设计。能够满足当前负载的架构,在请求数量、状态规模或服务实例增加后,未必还能达到原有目标。因此,分析架构演进时,需要判断原有方案在什么条件下开始失效,以及什么机制能够解决随之出现的新问题。

系统通常不会在项目初期就采用复杂的分布式架构。它往往从集中式结构起步,先建立最小的在线闭环;当并发请求成为瓶颈后,再提高单机的并发处理能力;当单机资源达到上限后,再拆分到多台机器协作;当服务实例数量继续增长、人工运维难以支撑时,再把部署和运行管理交给平台持续治理。每一次演进都对应上一阶段已经暴露的容量、状态协作或运维问题。

下面以在线游戏服务端为例展开这一演进过程。系统从只支持两名玩家的双人对战起步,随着玩家数量和系统规模增长,依次面对并发连接排队、单机性能耗尽、多节点状态协作以及实例运维等问题。每往前推进一步,都会遇到新的瓶颈,也就要引入新的机制。

这四个阶段前后衔接,构成了全书后续章节的基本脉络。后续各章将分别深入每一个阶段的具体问题与实现方式,在线游戏服务端也将作为主要案例贯穿其中。当然,这些演进规律并不只适用于游戏,在线协作、电商平台和大模型服务遇到规模压力时,同样会经历类似的调整。

1.4.1 阶段一：建立最小在线闭环

最初阶段要验证请求能否完整地进入、处理、返回和保存。客户端能够建立通信通道,服务能够识别消息并执行规则,结果能够返回相关客户端,关键状态能够在进程退出后恢复。此时把接入、计算和状态访问放在一个服务中,能够减少跨进程通信和部署复杂度。

这一阶段的重点是明确协议规则和状态归属,而不是提前拆分服务。以在线游戏为例,一个简单的双人对战足以检验操作意图能否送达、服务器能否给出唯一结果、双方能否收到同步结果,以及结算记录能否可靠保存。

客户端只提交操作意图,单体服务集中完成消息解析、规则裁决和状态更新,再把统一结果返回相关客户端。需要跨会话保留的关键结果则写入进程之外的持久化存储。从请求进入、规则

裁决到状态更新和结果返回，这些环节首尾相接，构成一个最小的在线闭环，如图 3 所示。

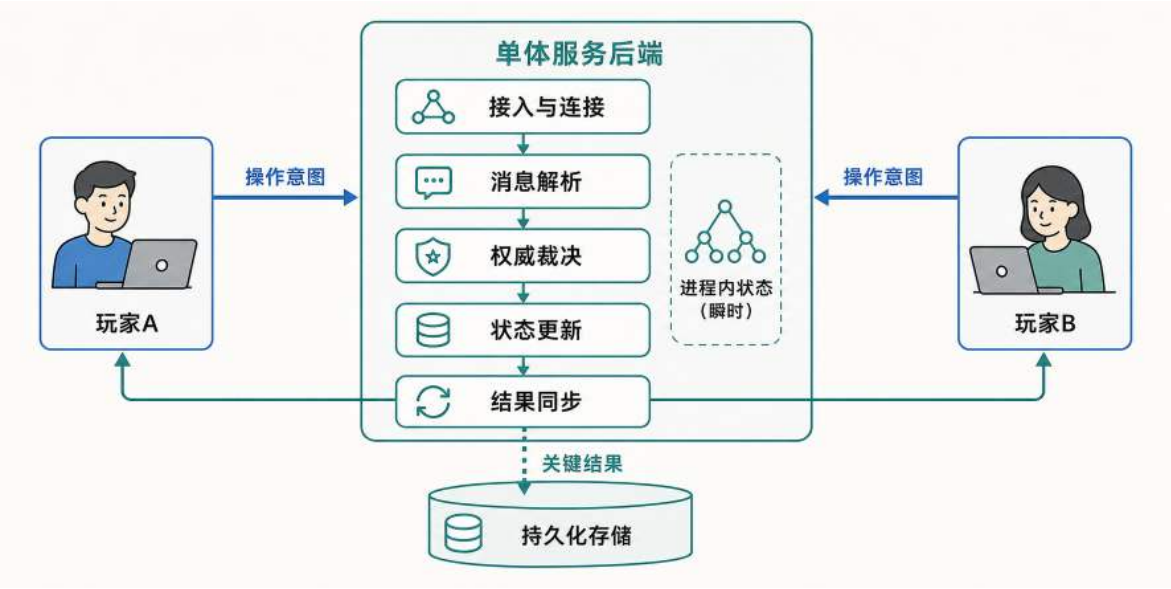


图 3: 最小在线闭环集中完成请求接入、权威裁决、状态更新和结果同步，并把关键结果保存到进程之外。

这一结构最核心的特征是集中：接入、裁决、状态更新和结果同步都在同一个服务进程内完成，只有需要跨会话保留的关键结果才落到进程之外的持久化存储中。它能够以较低复杂度验证核心流程，但关注的仍是一次请求如何完成，尚未说明多个请求同时到达时怎样共享有限的处理资源。

1.4.2 阶段二：提高单机并发能力

最小在线闭环完成后，系统开始承载更多用户，多个请求可能在相近时间到达。当前一个请求仍在等待网络、访问存储或执行计算时，新的请求已经进入系统。如果服务同一时刻只能处理一项请求，后续请求就只能在入口等待。当请求到达速度持续高于处理速度时，等待队列会不断增长，响应延迟也随之上升。

提高单机资源利用率的基本思路，是在一个请求等待外部资源时调度其他请求继续推进。多个请求同时推进后，系统还需要保护共享状态，避免并发修改产生数据竞争。连接池、对象池和缓存等资源复用机制可以进一步减少重复创建和远程访问开销。

这一阶段仍然使用单机资源，目标是在拆分系统之前识别真实瓶颈。请求到达后先进入队列，再由多个处理任务分别执行。这些同时推进的处理任务可以统称为并发执行体，它们访问共享状态时需要进行同步保护，缓存和对象复用则用于减少高频路径中的重复开销。由此形成单机并发处理过程，如图 4 所示。

并发处理的关键环节包括请求排队、多个执行体同时推进、共享状态处的同步保护，以及缓存和对象复用等资源优化手段；排队长度和尾延迟则是衡量系统是否接近饱和的重要信号。并发优化提高的是现有资源的利用效率，并没有增加机器本身的 CPU、内存、网络 and 连接容量。当

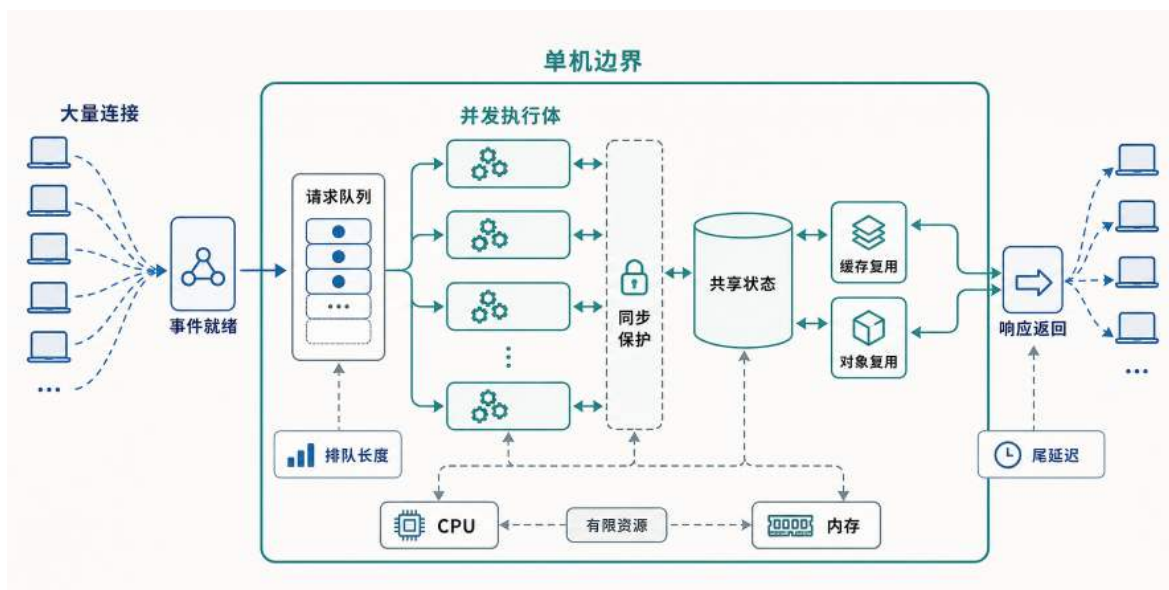


图 4: 单机并发通过请求排队、并发执行、共享状态保护和资源复用扩大处理能力,但仍受单机物理资源约束。

单机容量、故障影响范围或地域部署已经不能满足目标时,系统才需要引入多机拆分;资源接近上限只是其中最常见的触发条件。

1.4.3 阶段三：拆分到多台机器

单台机器的物理资源存在上限。系统扩展到多台机器后,请求、计算和数据必须按照某种规则分配到不同节点。拆分可以增加总容量并缩小单个节点的故障影响,但也会引入路由、状态迁移、副本同步和跨节点事务问题。

在游戏场景中,这种拆分通常按地图或玩家划分,跨地图移动对应状态迁移,跨服交互对应跨节点协调。对应的分片、复制、事务和共识机制,同样适用于订单、文档、任务和其他分布式状态。

服务入口按照状态归属把请求路由到不同计算节点,每个节点内部仍然依靠并发执行处理请求,只是状态被拆分成了不同分片。状态改变归属时,需要把相应数据从一个节点迁移到另一个节点。一次操作涉及多个分片时,节点之间需要通过网络同步必要信息。为了在节点失效时仍能继续服务,需要在备用节点上维持状态副本,由副本同步持续保持一致,故障发生后再由备用节点接管相应请求和状态责任。与此同时,原来在单机内部可以直接完成的调用变成了跨节点网络通信,网络延迟、消息丢失和部分失败成为必须面对的新代价。这些新增的路径、协作关系和代价共同构成多机协作结构,如图 5 所示。

多机拆分引入了三组新的机制和一组新的代价。三组机制分别是:按状态归属的分片路由决定请求去往哪个节点,状态迁移和状态同步处理归属变化与跨分片操作,副本同步与备用节点则在故障时接管相应责任。一组代价则来自跨节点通信:网络延迟使响应变慢,消息丢失需要重传来补救,部分失败则让操作结果不再像单机内部那样确定。横向扩展增加了可用资源,也缩小了单个节点的故障影响范围;代价是原来的进程内操作变成跨网络操作,状态正确性需要在延迟、

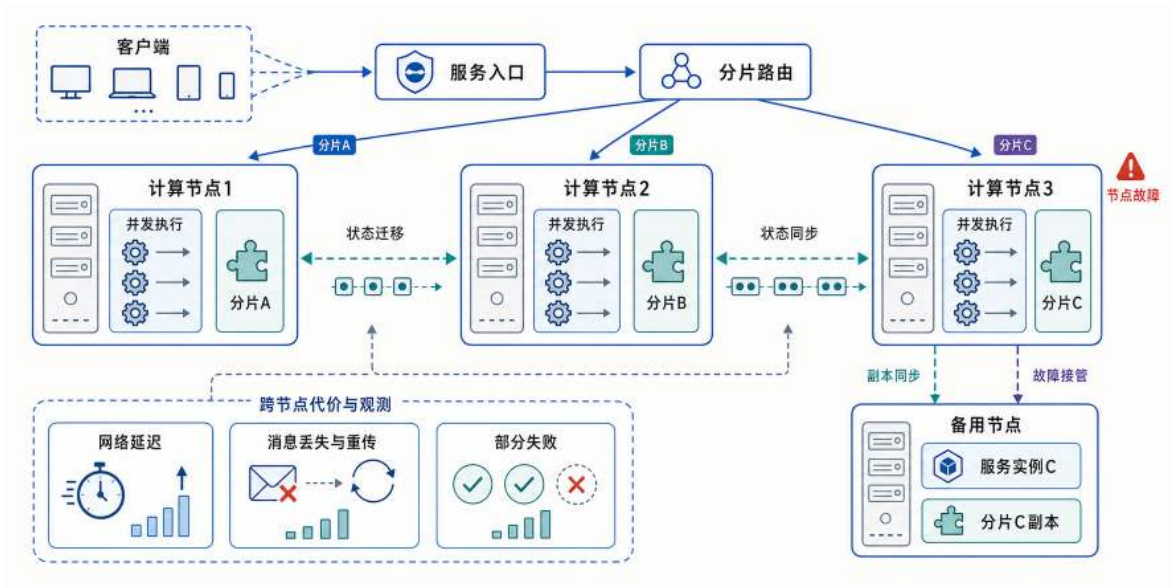


图 5: 多机拆分通过分片路由把请求分配到不同计算节点, 每个节点内部仍并发执行; 节点之间通过状态迁移和状态同步处理归属变化与跨分片操作, 副本同步为故障接管提供前提; 跨节点通信同时带来网络延迟、消息丢失与部分失败等新代价。

丢包和部分失败的情况下重新保证。

1.4.4 阶段四：交给平台持续治理

服务和节点数量增加后, 部署与运维本身会成为系统瓶颈。人工安装依赖、启动进程和替换故障实例难以保证一致性, 也无法及时响应频繁的负载变化。系统需要把运行环境封装为标准交付单元, 再由平台统一管理实例位置、数量、健康状态和访问入口。

平台依据声明的期望状态完成部署、调度、更新、故障恢复和弹性伸缩。这些自动化能力建立在容器隔离、虚拟网络和伸缩控制回路等机制之上。

应用声明版本、副本数量和资源需求后, 控制面持续比较期望状态与实际状态, 并通过调度创建、替换或回收运行实例。业务请求仍然经服务入口进入应用实例, 平台控制路径则在后台维持实例数量和健康状态, 由此形成持续调和过程, 如图 6 所示。

平台治理的核心是一个持续调和的控制回路: 控制面不断比较应用声明的期望状态与集群的实际状态, 通过调度执行缩小差异, 并在故障重建和弹性伸缩中反复这一过程。平台治理没有消除故障风险和容量限制, 而是把运行单元标准化, 并把原本依赖人工完成的部署、检查和恢复动作转化为可持续执行的控制过程。

四个阶段串起来看, 架构演进的本质不是结构越拆越复杂, 而是随着压力增加, 把原来集中在一起的职责逐步拆开, 用更多组件换取更大的容量、更好的隔离和更稳定的运行。每一次拆分都有明确的代价: 并发引入了状态竞争的风险, 多机拆分引入了网络延迟和部分失败, 平台治理则增加了系统的抽象层数和运维复杂度。没有哪一种架构天然更好, 选择的依据永远是当前最突出的瓶颈是什么, 以及愿意为新机制付出多少代价。

到这里为止, 我们讨论的接入、执行、状态、控制面和验证面, 以及它们的演进过程, 都还

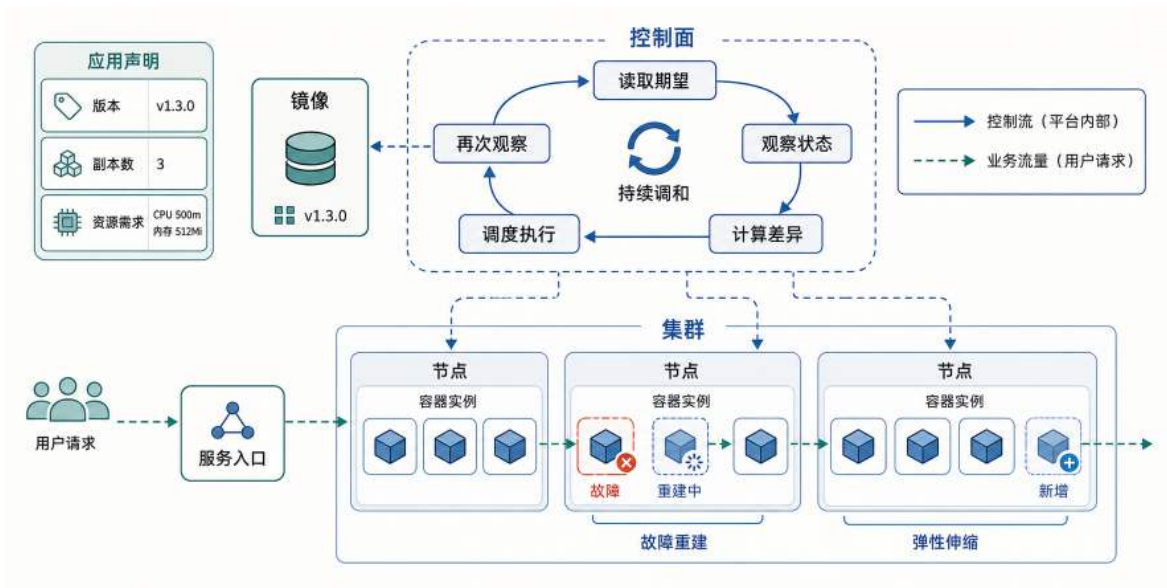


图 6: 平台通过持续调和维持应用声明的版本、容量和资源需求，并在实例故障或负载变化时执行重建与扩缩容。

停留在逻辑职责的层面。但这些职责最终都要运行在真实的计算、存储和网络设备上。逻辑上能成立的方案，落到物理资源上可能受限于带宽、延迟或硬件容量。下一节就把逻辑架构落到真实的基础设施上，看看物理条件如何约束架构的实际能力。

1.5 从逻辑架构到云基础设施

逻辑架构最终要运行在真实的计算、存储和网络资源之上。进程需要 CPU 时间和内存空间，状态需要磁盘或远程存储，服务之间的每次调用都要经过网络链路。逻辑结构能够独立扩展到什么程度，受到这些物理资源及其连接方式的约束。

要理解逻辑职责如何落在真实资源上，可以沿着一次请求的路径，看看它从终端到服务端经过了哪些物理环节。以游戏请求为例，操作指令从玩家设备出发，经过本地接入网络和骨干网络到达数据中心入口，再由内部网络转发到具体服务器和存储设备，这一完整路径如图 7 所示。其他在线系统的请求虽然内容不同，但也要经过相同类型的终端、网络入口、计算节点和状态存储。

数据中心内部由服务器、存储设备和网络设备承载业务处理，并依靠供电、制冷和监控系统维持运行。云平台进一步把具体硬件抽象成虚拟机、容器、存储卷、虚拟网络和托管服务，使应用能够按统一接口申请资源，而不必直接管理每台物理机器。

资源抽象带来三层逐步深化的能力。最基础的是资源池化，多项业务共享同一批硬件，并按需求获得不同配额。在此之上，运行环境可以依据相同镜像和配置在不同机器上重新创建；对状态服务而言，新实例能否继续工作还取决于状态是否已经外置或能够恢复。更进一步，实际运行状态可以被持续监测和调整，控制器能够根据故障和负载变化执行替换或扩缩容——这正是前面讨论的控制面与验证面在资源层的实现基础。

这些能力仍然受物理条件限制。扩容需要找到有剩余 CPU 和内存的节点，镜像和数据需要经过网络传输，状态服务还要满足容量和恢复要求。云平台降低了资源管理的复杂度，但不会消

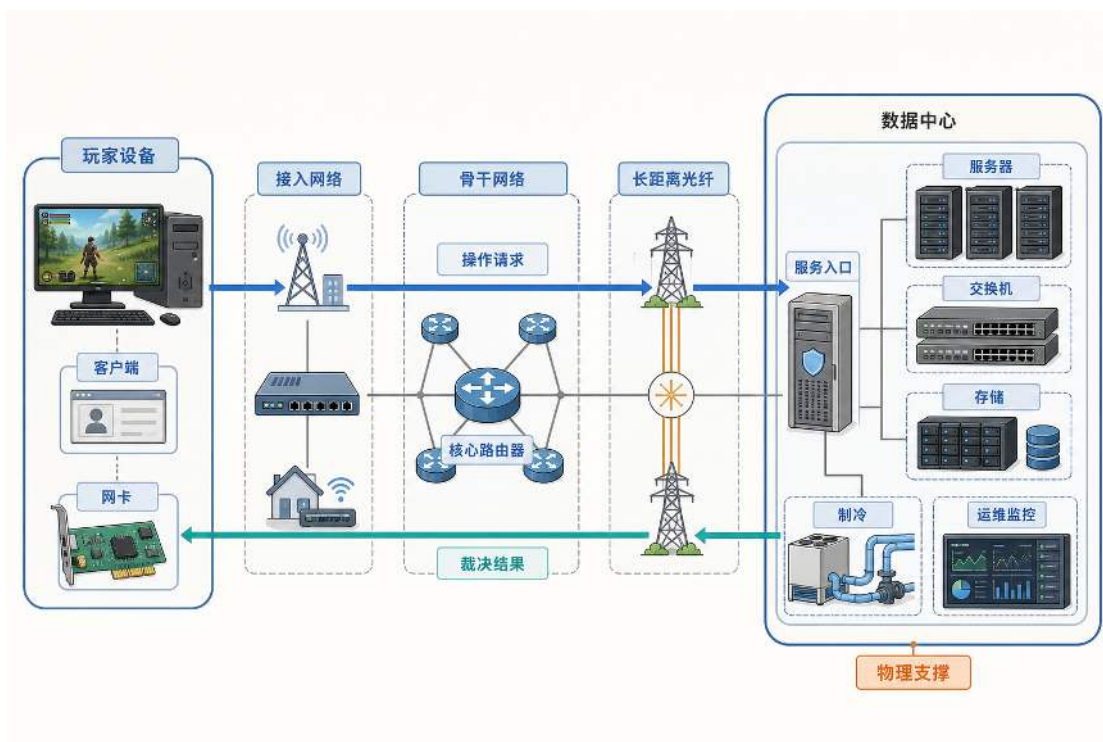


图 7: 在线请求在游戏案例中的基础设施路径：终端请求经过接入网络和骨干网络进入数据中心，再由计算、网络和存储资源共同完成处理。

除带宽、启动时间、存储延迟和故障恢复成本。理解这些限制，是正确使用云服务的前提。

逻辑职责在具体技术中的对应 资源抽象能力最终体现在，应用可以灵活选择承载各项逻辑职责的具体技术组件。以上讨论的接入、执行、状态、控制面和验证面都是逻辑职责，在实际系统中需要由具体技术来承载。不同业务场景可以选择不同的技术组合，但职责划分和权衡原则是相通的。

在线游戏案例中，逻辑职责与技术组件之间存在清晰的对应关系。沿请求路径从左到右，长连接接入由 WebSocket 协议承载，服务执行由 Go 编写的应用服务完成，服务运行环境则打包为 Docker 镜像以保证可复制。高频运行状态保存在内存缓存 Redis 中，需要持久保留的记录则写入 PostgreSQL 等关系型数据库。控制面由 Kubernetes 承担，完成部署调度和故障替换等治理动作。验证面则通过运行指标、事件日志和链路记录三种方式，从各层收集运行证据。这些对应关系如图 8 所示。

这张图把前面讨论的五层逻辑职责落到了具体的技术组件上：沿请求路径，接入层由 WebSocket 承载、服务执行层运行 Go 服务、状态层用 Redis 和 PostgreSQL 分别保存运行状态与持久记录；控制面由 Kubernetes 完成部署调度与故障替换；验证面通过指标、日志和链路记录收集各层运行证据。读者可以顺着请求路径对照正文中学到的每一层职责，找到对应的技术实现。

需要强调的是，技术选择是可变的，但是职责划分是稳定的。接入协议可以从 WebSocket 换成 HTTP 或 gRPC，执行服务可以用 Java、Python 或其他语言编写，状态存储可以换成 MySQL、MongoDB 或云厂商的托管数据库，控制面可以用 Kubernetes 也可以用其他容器编排系统，验

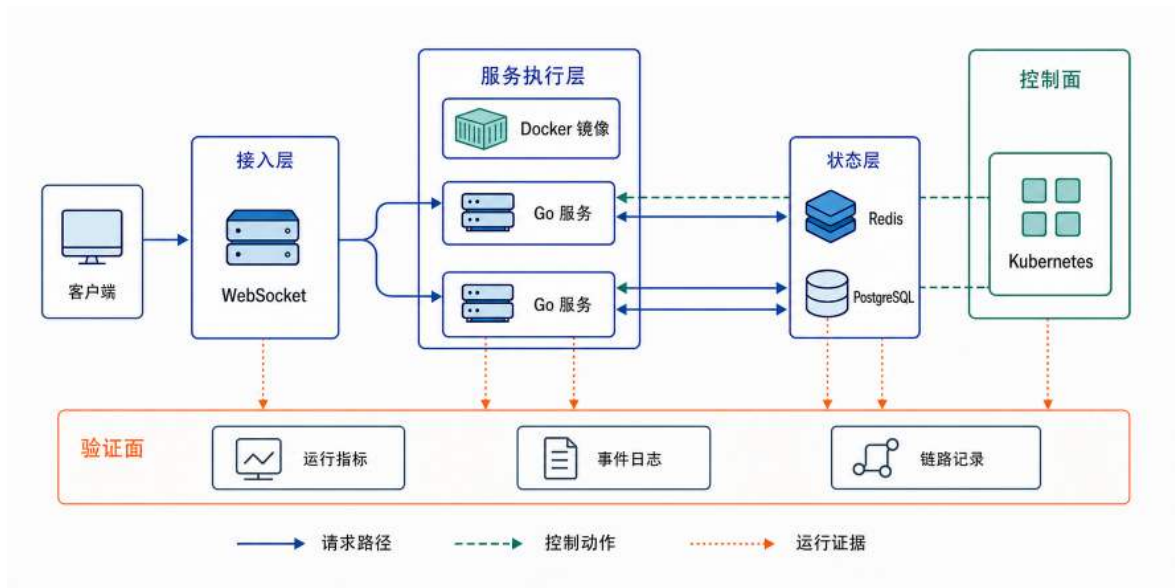


图 8: 逻辑职责在技术栈中的承载方式：接入层、服务执行层和状态层构成主要请求路径，控制面负责交付与治理，验证面从各层采集指标、日志和链路记录。

证面的工具也可以从不同方案中选择。

说到底，图里的每一种技术都只是某个职责的具体载体。在线系统真正稳定的东西，不是用了什么框架、什么数据库，而是请求处理、状态维护、运行治理和可验证性这几组恒常的问题。技术随着时间会一直向前迭代，但问题和职责不会轻易改变。

1.6 小结

在线系统持续运行时，需要同时应对四方面的挑战：规模增长带来的并发压力、共享状态的正确性要求、负载变化下的资源分配，以及故障与运维的自动化需求。接入层、服务执行层和状态层构成主要请求路径，控制面负责部署、调度和故障恢复，验证面通过指标、日志和链路记录判断系统是否正常运行。主要请求路径完成一次业务处理，控制面和验证面让这条路径在负载变化与组件故障下持续运行，所有职责最终都依赖计算、存储和网络资源支撑。

性能、状态正确性、可扩展性、可用性和成本，是评价架构设计的五个基本维度。它们不是孤立的目标，而是彼此牵制：更严格的状态确认可能增加延迟，更多副本会提高可用性但增加成本，更高的资源利用率会压缩应对故障的余量。这些目标共同推动系统从最小闭环走向单机并发、多机协作和平台治理。每一次演进都应当对应已经暴露的容量、状态、故障或运维限制，新增机制的作用也要用具体指标和故障场景来验证。

本章给出的是一套分析在线系统的基本方法：先沿着请求路径拆分职责，再用一组评价维度判断设计的好坏，最后顺着压力增长的方向看架构如何演进。这套方法并不依赖某一种业务或技术栈，换一个场景，职责划分和判断逻辑仍然成立。

后续章节就沿着这条思路，以在线游戏服务端为主要案例，按照压力出现的顺序逐层展开。第 2 章“双雄对战”建立最小网络闭环，讨论通信通道、消息边界和状态裁决。第 3 章“英雄集结”处理单机并发、共享状态保护和资源复用。第 4 章“切分世界”进一步将计算和状态拆到多

台机器，讨论多机分片、跨节点协作、一致性与容错。第 5 章“飞升入定”把服务封装并交给集群平台管理，讨论容器化交付、编排与弹性运行。第 6 章“穷理尽微”则下沉到平台内部，分析容器、网络和弹性控制的底层实现机制，并把这些能力迁移到高并发 Sandbox 场景。

1.7 思考题

1. 选择在线协作、电商或大模型服务中的一种系统，划分其客户端、接入层、服务执行层、状态层、控制面与验证面职责，并说明关键状态由谁最终确认。
2. 登录请求通常可以分配给新启动的服务实例，已经开始的对局或协作会话却不能直接交给任意实例。分别说明两类工作横向扩展所需的状态条件，并给出验证扩展是否有效的指标。
3. 假设一个服务实例突然失效，设计从故障发现到服务恢复的完整过程。指出哪些状态可能丢失，以及缩短恢复时间会增加哪些成本。
4. 比较“始终按峰值准备容量”和“根据负载弹性调整容量”两种方案。分析它们在性能、弹性和成本上的差异，并说明启动时间与排队长度会怎样影响选择。
5. 比较在线游戏与大模型服务在连接持续时间、状态归属、计算资源和扩容信号上的差异。哪些架构判断可以直接迁移，哪些机制需要根据负载特征调整？